

Migración de GBSwagger a SwagDoc

Esta guía migra una API escrita con [Horse](#) de [GBSwagger](#) a SwagDoc, que escribe la documentación como un documento OpenAPI 3 en lugar de uno Swagger 2.0.

Qué cambia

	GBSwagger	SwagDoc para Horse
Especificación	Swagger 2.0	OpenAPI 3 o Swagger 2.0, por la propiedad <code>SpecVersion</code>
Documentación	Atributos en los controllers y en los models	Objetos del documento, escritos en código
Rutas	Registradas por los atributos, en el registro de paths	Registradas en Horse por la aplicación
Interfaz de usuario	Swagger UI	Swagger UI o Scalar
Archivos de la interfaz	Cargados desde una dirección externa	Cargados desde una dirección externa o incrustados

La diferencia más importante es la tercera. En GBSwagger los atributos `SwagGET`, `SwagPOST` y `SwagPath` **también registran las rutas** en Horse, así que quitarlos quita las rutas de la API. La migración tiene, por lo tanto, dos partes: registrar las rutas y documentarlas.

1. Dependencias

En el archivo `boss.json`, reemplace la dependencia:

```
- "github.com/gabrielbaltazar/gbswagger": "^3.0.7",  
+ "github.com/marcelojaloto/SwagDoc": "^2.0.0",
```

En el search path del proyecto, reemplace las tres carpetas de GBSwagger por las dos de SwagDoc:

```
..\..\modules\gbswagger\Source\Horse;..\..\modules\gbswagger\Source\Core;..\..\modules\gbswagger\Source\Validator;  
+..\..\modules\SwagDoc\Source;..\..\modules\SwagDoc\Integrations\Horse\Source;
```

2. El middleware

```
- Horse.GBSwagger,  
+ Horse.SwagDoc,
```

```

- THorse.Use(HorseSwagger('api/help'));
+ SwagDocConfig.UserInterfaceRoute := '/api/help';
+ SwagDocConfig.DocumentRoute := '/api/help/openapi.json';
+ SwagDocConfig.DiscoverRoutes := False;
+ THorse.Use(HorseSwagDoc);

```

`DiscoverRoutes` escribe las rutas registradas en Horse que todavía no están documentadas. Es útil mientras la migración no termina, para ver lo que falta, y se desactiva cuando todas las operaciones están documentadas. Tenga en cuenta que el descubrimiento compara la ruta registrada con la ruta del documento: cuando el documento declara un server con una ruta base, como `http://localhost:9000/api`, las rutas del documento no repiten el prefijo `/api` y todas parecen no documentadas para el descubrimiento.

3. Las rutas

Los atributos de GBSwagger registran las rutas, así que ahora las registra la aplicación. Un controller cuyos métodos responden a las rutas sigue funcionando con un método de clase que lo crea, llama al método y lo destruye:

```

class procedure TController.Execute(Request: THorseRequest; Response: THorseResponse;
  const Action: TControllerAction);
var
  Controller: TController;
begin
  Controller := Self.Create(Request, Response);
  try
    Action(Controller);
  finally
    Controller.Free;
  end;
end;

```

El constructor del controller debe ser `virtual`, para que `Self.Create` cree la clase de la ruta. Con él, cada ruta se registra así:

```

THorse.Get('/api/tasks',
  procedure(Request: THorseRequest; Response: THorseResponse)
  begin
    TTaskController.Execute(Request, Response,
      procedure(Controller: TController)
      begin
        TTaskController(Controller).List;
      end);
  end);

```

La sección `initialization` que registraba el controller en GBSwagger se elimina:

```

-initialization
- THorseGBSwaggerRegister.RegisterPath(TTaskController);

```

4. La documentación de las operaciones

Cada atributo pasa a ser una propiedad de los objetos de SwaggerDoc. El método `Route` traduce una ruta escrita con la sintaxis de Horse y agrega un parámetro por cada variable de la ruta.

GBSwagger	SwaggerDoc
[SwaggerPath('tasks', 'Tasks')]	SwaggerDocApi.Route('/tasks') y Operation.Tags.Add('Tasks')
[SwaggerGET('List of all tasks')]	Path.AddOperation(ohvGet) y Operation.Summary
[SwaggerPOST('Create a new task')]	Path.AddOperation(ohvPost)
[SwaggerPUT('{id}', 'Change data')]	SwaggerDocApi.Route('/tasks/{id}').AddOperation(ohvPut)
[SwaggerPATCH(...)] , [SwaggerDELETE(...)]	ohvPatch , ohvDelete
[SwaggerPathParam('id', 'Task Id')]	Escrito por <code>Route</code> ; la descripción se define en <code>Path.Parameters[0]</code>
[SwaggerParamQuery('count', 'Description', False, False)]	TSwagRequestParameter con <code>InLocation := rpiQuery</code>
[SwaggerParamBody('Task data', TTaskModel)]	Operation.RequestBody.AddMediaType('application/json').Schema.Name := 'task'
[SwaggerResponse(200, TTaskModel, 'Task list')]	TSwagResponse con <code>StatusCode</code> , <code>Description</code> y el media type
[SwaggerResponse(404)]	TSwagResponse con la descripción del estado

Una operación documentada con SwaggerDoc:

```
vOperation := SwaggerDoc.Api.Route('/tasks/{id}').AddOperation(ohvGet);
vOperation.OperationId := 'getTask';
vOperation.Summary := 'Get data for a specific task';
vOperation.Tags.Add('Tasks');

vResponse := TSwaggerResponse.Create;
vResponse.StatusCode := '200';
vResponse.Description := 'Task data';
vResponse.AddMediaType('application/json').Schema.Name := 'task';
vOperation.Responses.Add(vResponse.StatusCode, vResponse);
```

Las operaciones escritas en código son más largas que los atributos y, a cambio, alcanzan todo el modelo de la especificación: request bodies con varios media types, ejemplos, links, callbacks, webhooks y los objetos reutilizables de los components.

5. Los schemas

Los atributos de los models se reemplazan por schemas escritos con `TJsonSchema` y agregados al documento como definitions. SwaggerDoc los escribe bajo `definitions` en un documento Swagger 2.0 y bajo `components/schemas` en uno OpenAPI 3.

GBSwagger	SwaggerDoc
[SwaggerProp('title', 'Description', True, False)]	AddField<string>('title', 'Description') con Required
[SwaggerString(100)]	TJsonFieldString con MaxLength
[SwaggerString(36, 36)]	TJsonFieldString con MinLength y MaxLength
[SwaggerNumber(0, 4)]	TJsonFieldInteger con MinValue y MaxValue
Una propiedad de un tipo clase	AddField<T> del tipo, o un \$ref a otro schema

```
vSchema := TJsonSchema.Create;
vSchema.Root.Description := 'A task of the manager';

vTitle := TJsonFieldString(vSchema.AddField<string>('title', 'Task title description.'));
vTitle.Required := True;
vTitle.MaxLength := 100;

vDefinition := TSwaggerDefinition.Create;
vDefinition.Name := 'task';
vDefinition.JsonSchema := vSchema.ToJson;
SwaggerDoc.Api.Definitions.Add(vDefinition);
```

Un schema escrito a mano referencia otro con `#/components/schemas/<nombre>` en un documento OpenAPI 3 y con `#/definitions/<nombre>` en uno Swagger 2.0. Las aplicaciones que publican las dos versiones eligen el prefijo por la propiedad `SpecVersion` del documento.

6. La seguridad

```
- Swagger
- .AddBearerSecurity
- .AddCallback(HorseJWT(TJWTSettings.SECRET_KEY))
```

El middleware del token lo agrega la aplicación, en las rutas que protege:

```
THorse.Use('/api/tasks', HorseJWT(TJWTSettings.SECRET_KEY));
```

Y el documento declara el esquema y las operaciones que lo exigen:

```
vBearer := TSwagSecurityDefinitionHttp.Create;
vBearer.SchemeName := 'bearerAuth';
vBearer.Scheme := 'bearer';
vBearer.BearerFormat := 'JWT';
SwagDocApi.SecurityDefinitions.Add(vBearer);

vRequirement := vOperation.AddSecurityRequirement;
vRequirement.AddScheme('bearerAuth', []);
```

7. La información de la API

```
- Swagger
- .Info
- .Title('Tasks API')
- .Description('The API aims to manage information about tasks.')
- .Contact
- .Name('Marcelo Jaloto')
- .&End
- .&End
- .BasePath('api')
- .AddProtocol(TGBSwaggerProtocol.gbHttp)
+ SwagDocApi.Info.Title := 'Tasks API';
+ SwagDocApi.Info.Description := 'The API aims to manage information about tasks.';
+ SwagDocApi.Info.Contact.Name := 'Marcelo Jaloto';
+ SwagDocApi.AddServer('http://localhost:9000/api', 'Local server');
```

En OpenAPI 3 el host, la ruta base y el protocolo se reemplazan por los servers. Una aplicación que sigue publicando un documento Swagger 2.0 usa las propiedades `Host`, `BasePath` y `Schemes`, y `SwagDoc` las traduce a un server cuando el documento se escribe como OpenAPI 3.

8. La versión de la especificación

El documento publicado por el middleware se escribe como OpenAPI 3. Una API que necesita mantener el documento Swagger 2.0 durante la migración cambia una propiedad:

```
SwagDocApi.SpecVersion := svSwagger2;  
SwagDocConfig.DocumentRoute := '/api/help/swagger.json';
```

Los mismos objetos producen los dos documentos, así que las dos versiones se pueden comparar antes de abandonar la antigua.

Checklist

- ☐ La dependencia y el search path apuntan a `SwagDoc`.
- ☐ Todas las rutas que registraban los atributos están registradas en `Horse`.
- ☐ Todas las operaciones están documentadas, y `DiscoverRoutes` ya no muestra nada.
- ☐ Las rutas del documento corresponden a las registradas, considerando la ruta base del server.
- ☐ Los models ya no usan los atributos de `GBSwagger`.
- ☐ El middleware del token protege las rutas que exigían la seguridad.
- ☐ El documento es válido en las dos versiones, si la API publica las dos.

Un ejemplo completo

El ejemplo [tasks-manager-horse](#) se migró con estos pasos: siete operaciones, cinco schemas, autenticación con JWT y un documento OpenAPI 3.2.1.